



CMC UNIVERSITY

Aspire to Inspire the Digital World

Machine Learning and Data Mining

Faculty of Information and Communication Technology

2.1. Convolutional Neural Network

2.2 Recurrent Neural Network

2.3 Long-Short Term Memory

2.4 Optimization techniques

Optimization techniques

Learning Objectives

- Understand the mathematical foundations of optimization in Deep Learning.
- Compare various optimization algorithms (SGD, Momentum, Adam).
- Master stabilization techniques like Weight Initialization and Batch Normalization.
- Implement regularization methods, specifically Dropout, to prevent overfitting.

The Optimization Landscape

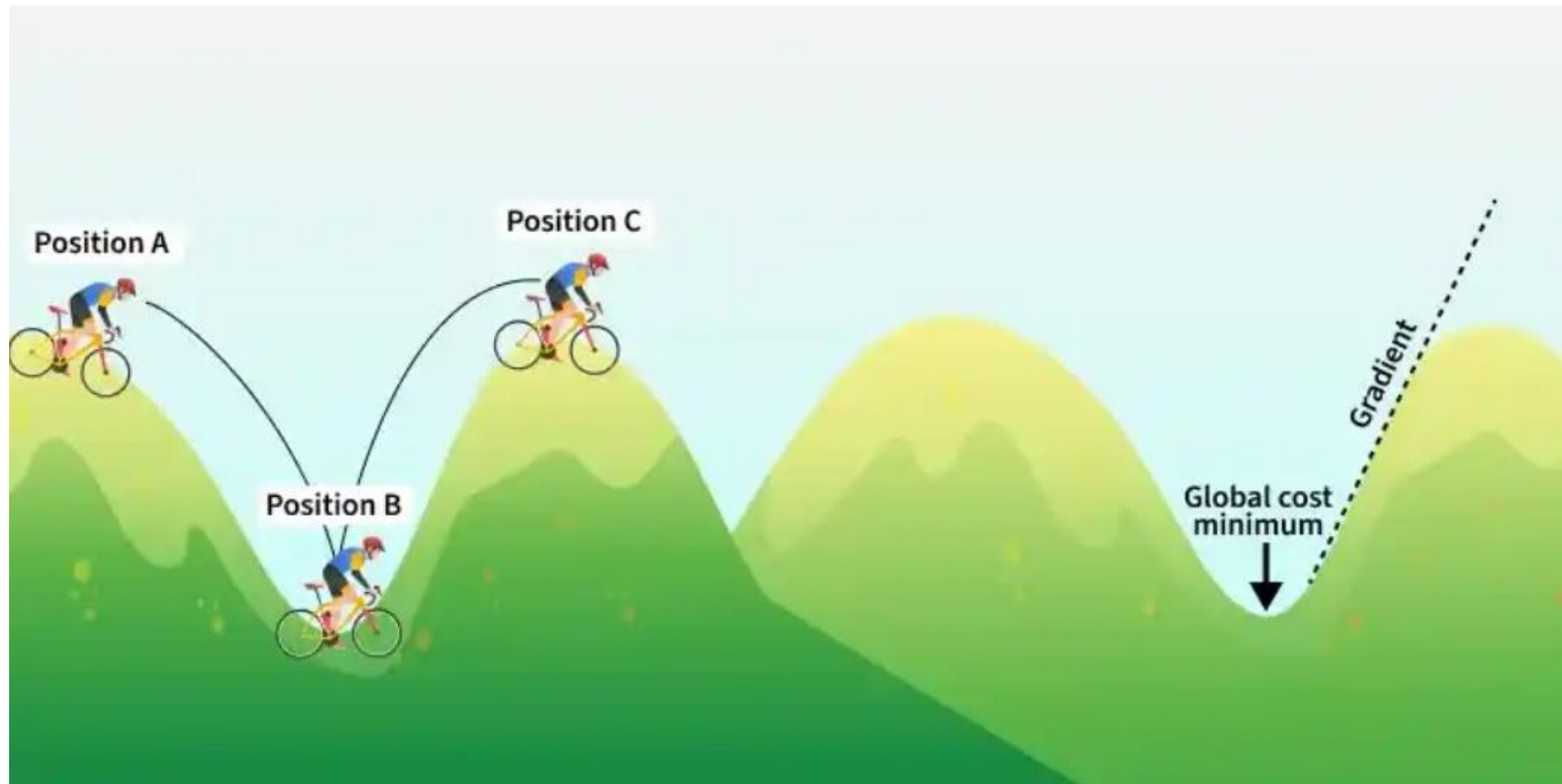
- Goal: Minimize the Loss Function $L(w)$ with respect to parameters w .
- Optimization vs. Generalization: Finding a global minimum on training data vs. performing well on unseen data.

Gradient Descent Foundations

The update rule: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla L(\mathbf{w}_t)$.

η (Learning Rate): The most critical hyperparameter.

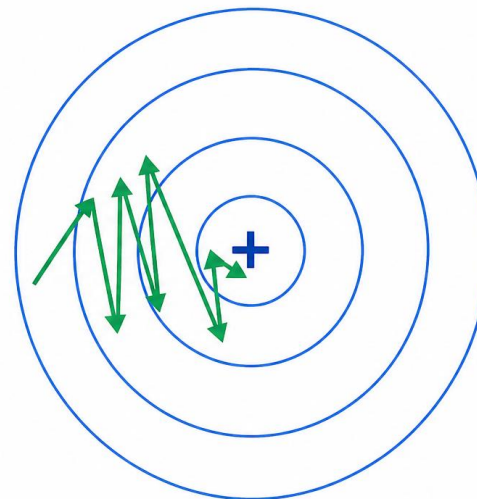
Visualizing gradient descent on a loss surface.



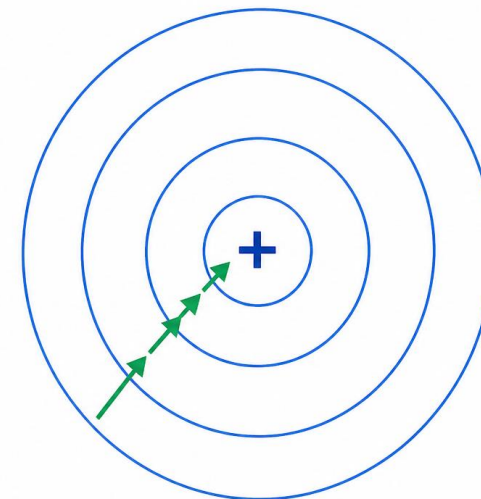
Stochastic Gradient Descent (SGD)

- Problem with Batch Gradient Descent: Computationally expensive for large datasets.
- SGD Solution: Update parameters using a single sample or a mini-batch.
- Efficiency and the introduction of "noise" that helps escape local minima.

Stochastic Gradient Descent



Gradient Descent

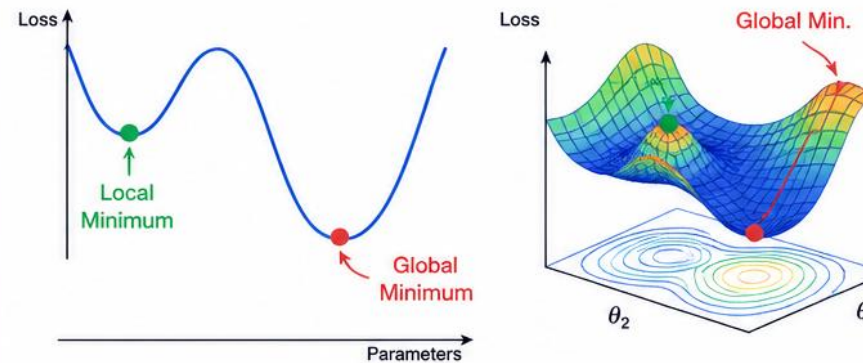


Challenges in Optimization

- Local Minima vs. Global Minima.
- Saddle Points: Areas where the gradient is zero but not a local extremum.
- Vanishing and Exploding Gradients in deep architectures.

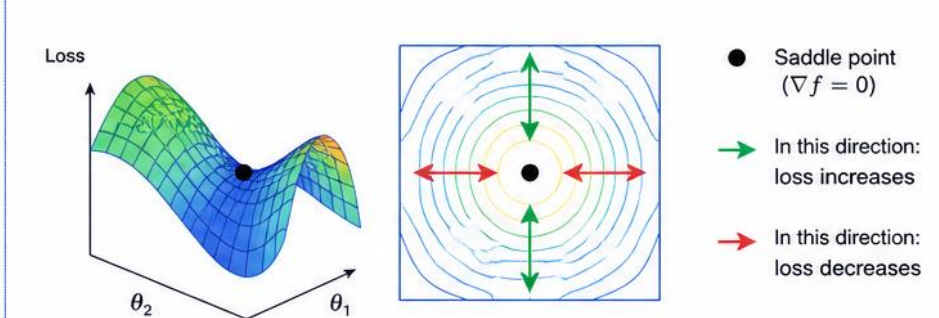
Challenges in Optimization

1. Local Minima vs. Global Minima



Optimization can get stuck in a local minimum, which is not the best solution.

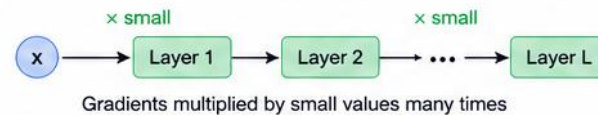
2. Saddle Points



At saddle points, the gradient is zero, but it is not a local minimum or maximum.

3. Vanishing and Exploding Gradients in Deep Architectures

Vanishing Gradients



Gradients $\rightarrow 0$

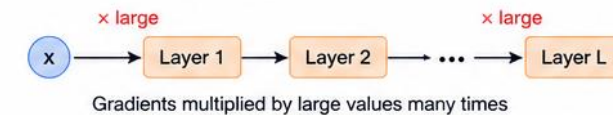
- Early layers learn very slowly.
- Model struggles to capture long-term dependencies.

Example causes:

Sigmoid/tanh activations, improper initialization, etc.

Backpropagated Gradients

Exploding Gradients



Gradients $\rightarrow \infty$

- Updates become too large.
- Training becomes unstable or diverges.

Example causes:

ReLU with large weights, improper initialization, etc.

Key Takeaway



These challenges make optimization in deep learning hard. Techniques like better initialization, normalization, residual connections, gradient clipping, and adaptive optimizers help address them.

Common Solutions



- Better initialization (He, Xavier)
- Normalization (BatchNorm, LayerNorm)

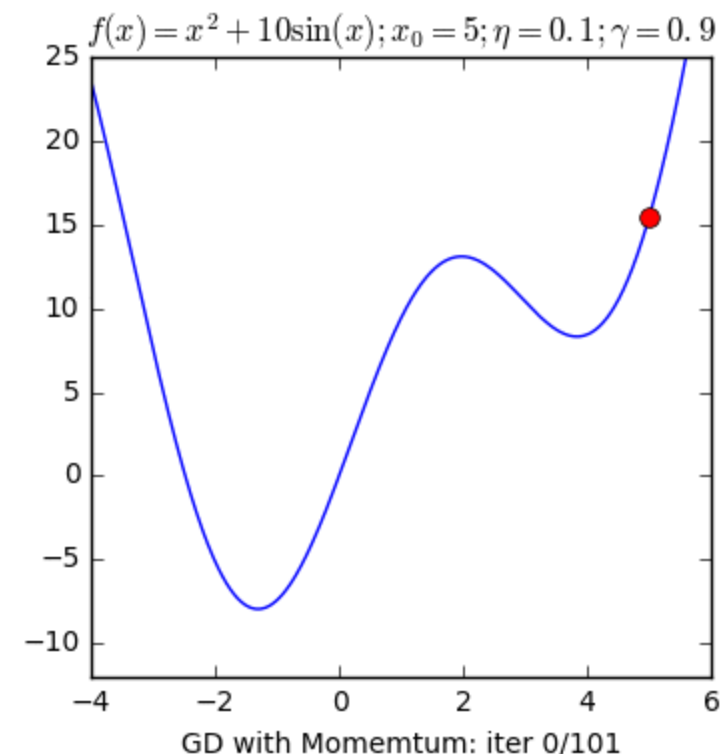
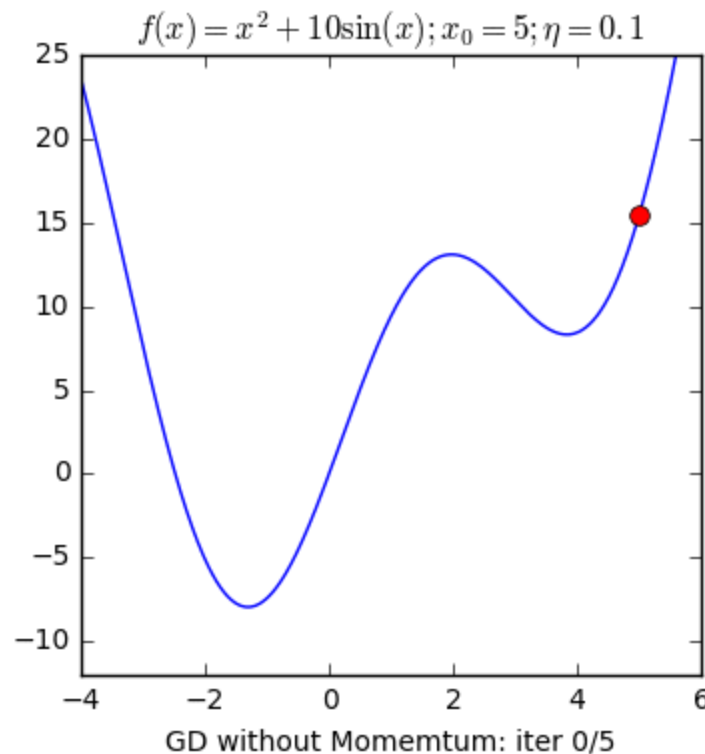
- ReLU / Leaky ReLU / GELU
- Gradient Clipping (for exploding gradients)
- Adaptive optimizers (Adam, RMSprop, etc.)

Momentum-based Optimization

Intuition: A heavy ball rolling down a hill gathers momentum.

Formula: $v_t = \beta v_{t-1} + (1 - \beta) \nabla L(\mathbf{w}_t)$; $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta v_t$.

Reduces oscillations and speeds up convergence in high-curvature directions.

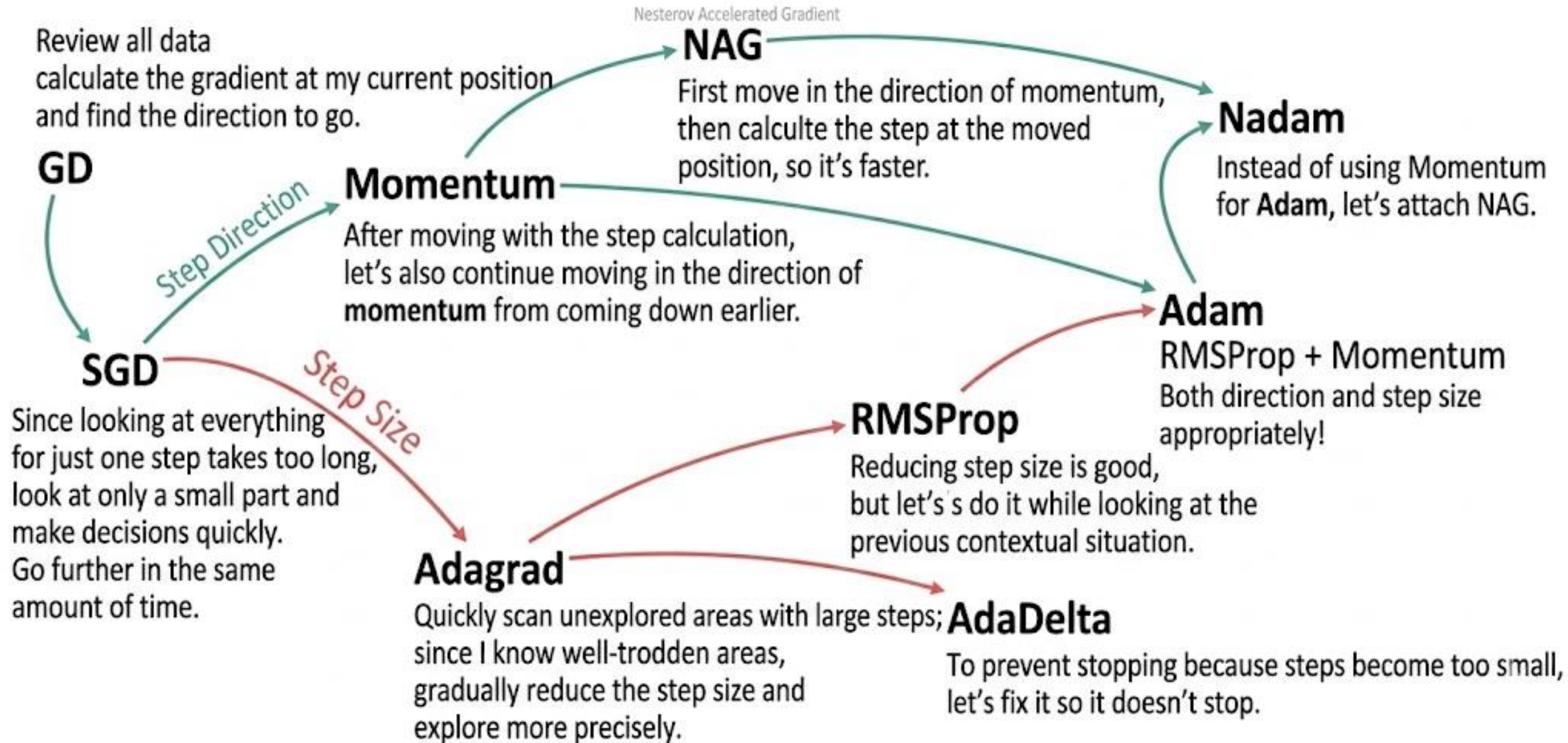


Adaptive Learning Rates: RMSProp

- Adjusts the learning rate for each parameter individually.
- Normalizes the gradient by a running average of its recent magnitudes.
- Prevents gradients from becoming too large or too small.

Adam Optimizer

- Combines the benefits of Momentum and RMSProp.
- Currently the "gold standard" for training deep neural networks.
- Key parameters: β_1 (momentum) and β_2 (scaling).



Learning Rate Scheduling

- Why keep η constant? Start large to explore, end small to fine-tune.
- Strategies: Step Decay, Exponential Decay, and Cosine Annealing.
- Warmup: Increasing the learning rate at the start of training to stabilize early updates.

Numerical Stability & Initialization

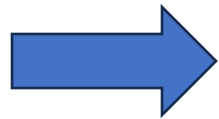
- The dangers of 0 or constant initialization (Symmetry breaking problem).
- Xavier (Glorot) Initialization: Keeps variance consistent across layers (ideal for Sigmoid/Tanh).
- He Initialization: Optimized for ReLU activation functions.

Batch Normalization (Concept)

- Internal Covariate Shift: The distribution of layer inputs changes during training.
- Solution: Normalize the activations of each layer within each mini-batch.

Batch Normalization (Mechanism)

- Calculate mean μ and variance σ^2 for the mini-batch.
- Normalize: $\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + e}}$
- Scale and Shift: $y = \gamma \hat{x} + \beta$ (parameters γ, β are learned).



- *Calculate mean and variance across the entire batch*
- *Standardize each feature*

Benefits of Batch Normalization

- Allows higher learning rates.
- Reduces sensitivity to weight initialization.
- Acts as a form of mild regularization.

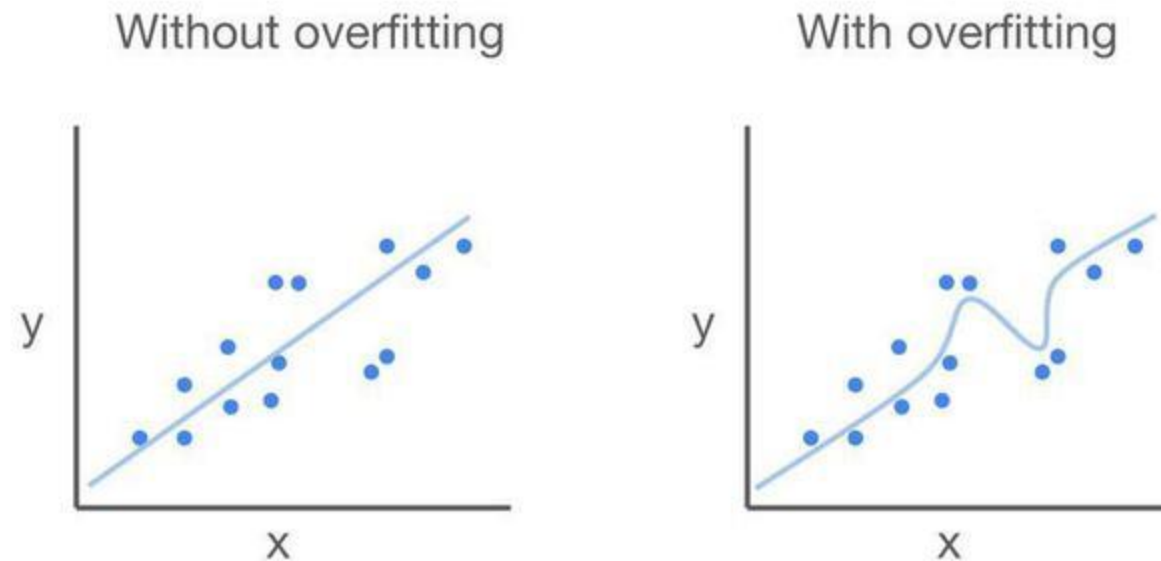
Layer Normalization vs. Batch Normalization

- Batch Norm depends on batch size (problematic for small batches).
- Layer Norm: Normalizes across the features of a single sample (standard in Transformers).

Criteria	Batch Normalization	Layer Normalization
Standardized according to	Batch	Feature (per sample)
Depends on batch size	Yes	No
Works well when	Large Batch	Every Batch
Used in	CNN, CV	Transformer, NLP
Train vs Inference	Different (moving avg)	Same

The Problem of Overfitting

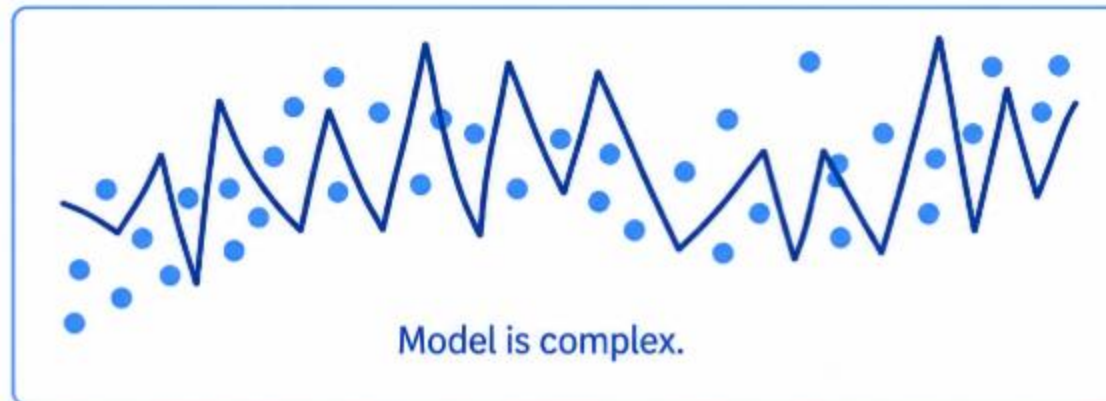
- High training accuracy but low validation/test accuracy.
- The model "memorizes" noise rather than learning patterns.



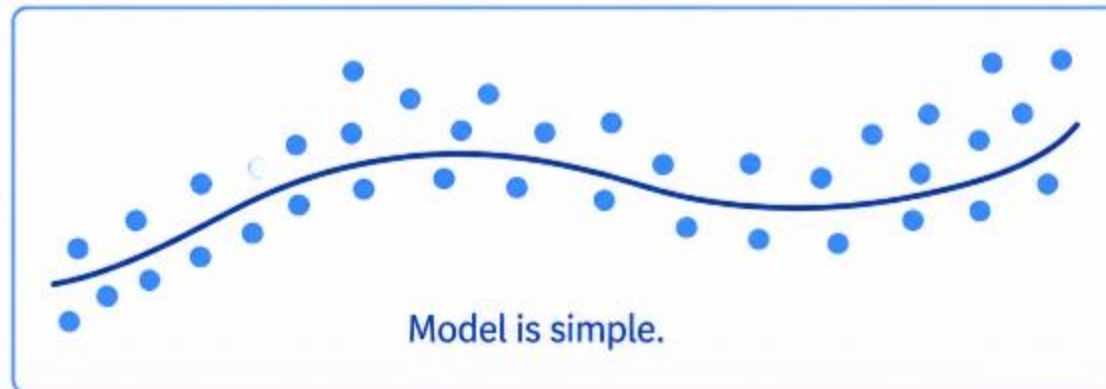
L2 Regularization (Ridge Regression)

- Adds a penalty term to the loss function: $L_{reg} = L + \frac{\lambda}{2} \|\mathbf{w}\|^2$.
- Forces weights to be small, leading to simpler models.

Overfitting
(No Regularization)

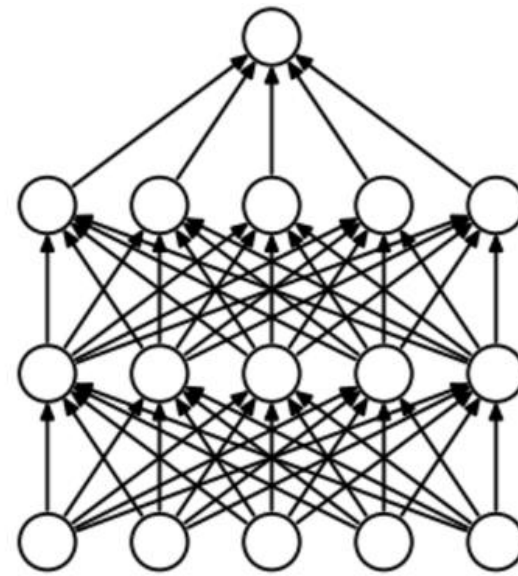


Balanced
(With Regularization)

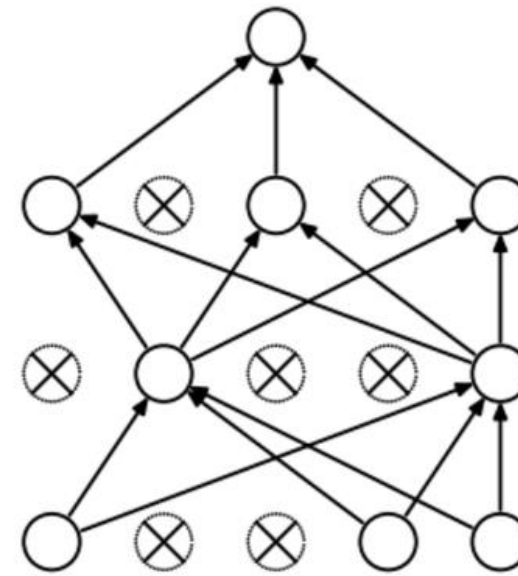


Dropout (Theoretical Framework)

- Concept: Randomly "drop" (set to zero) a fraction p of neuron activations during each training step.
- Forces the network to learn redundant representations (cannot rely on specific neuron).



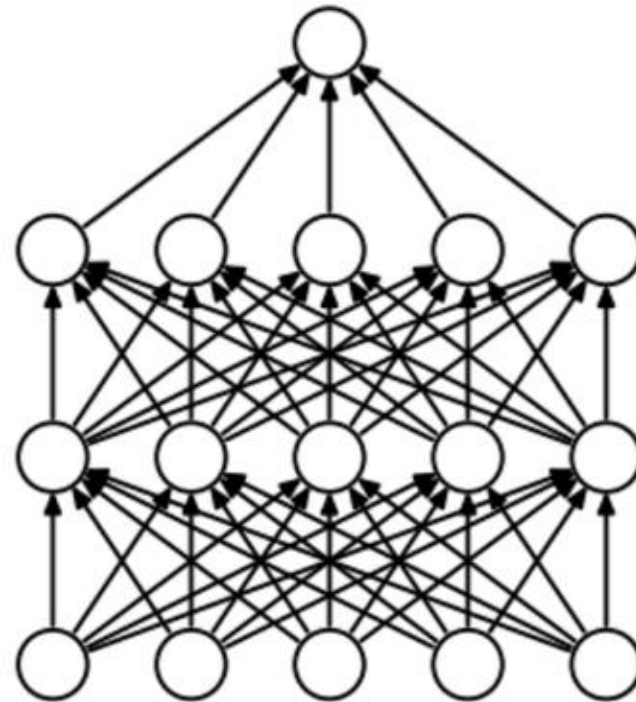
(a) Standard Neural Net



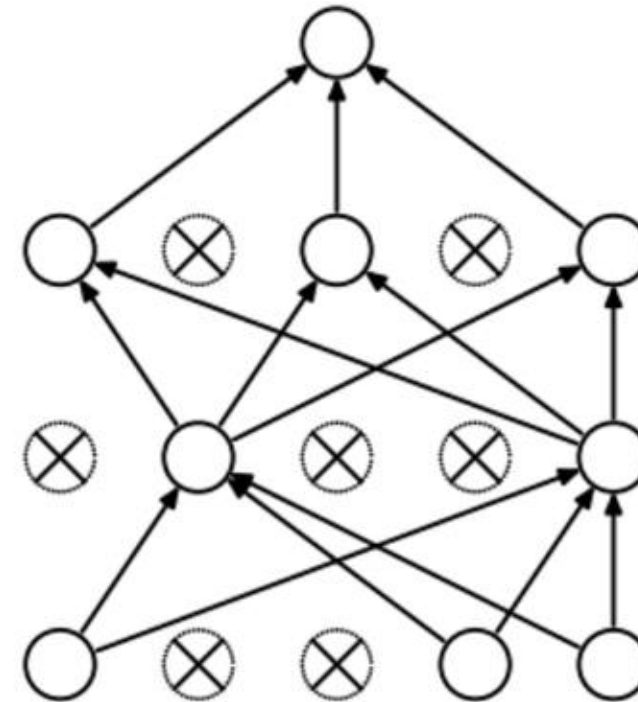
(b) After applying dropout.

Dropout (In Practice)

- **Training:** Neurons are dropped; remaining ones are scaled by $1/(1-p)$.
- **Inference:** All Neurons are used (no dropout) to ensure deterministic output.



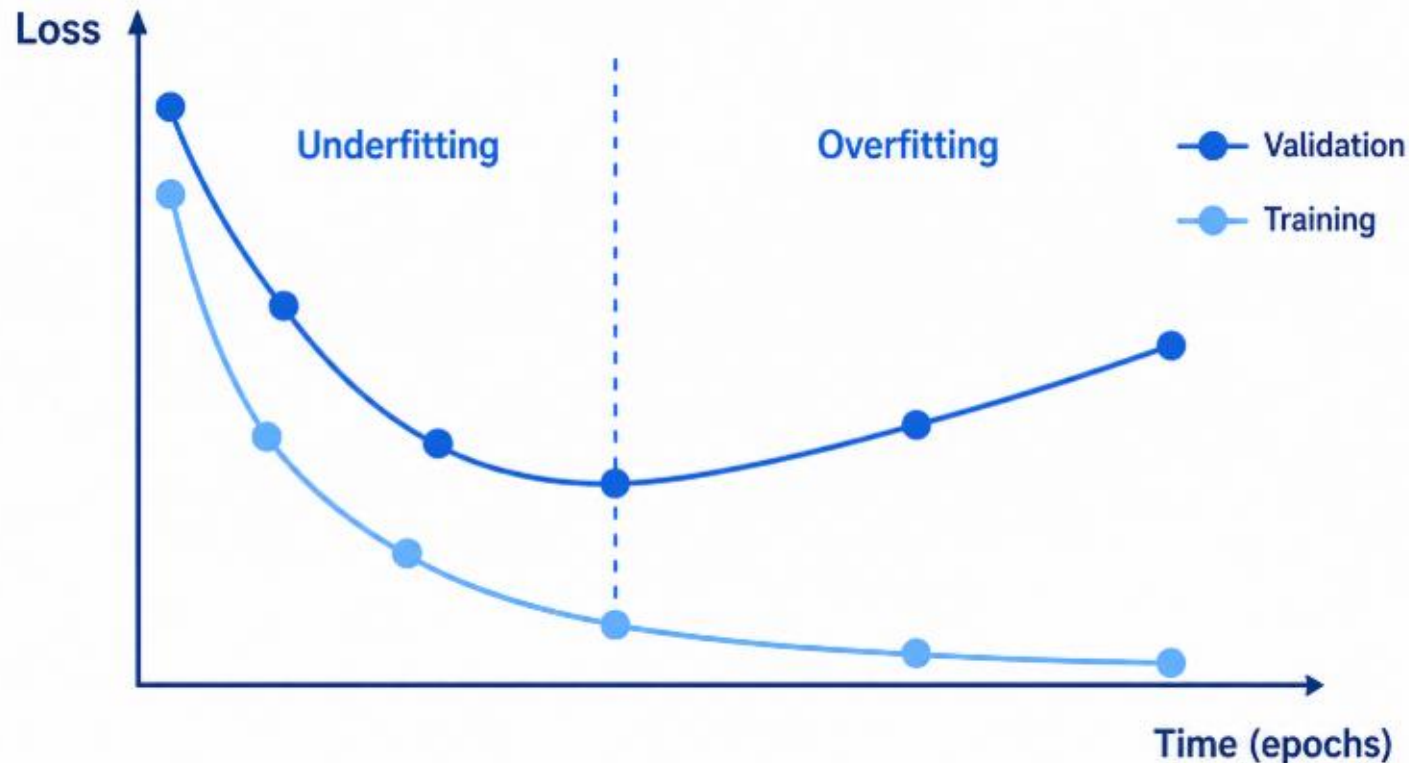
(a) Standard Neural Net



(b) After applying dropout.

Early Stopping

- Monitoring validation loss during training.
- Stop training when validation loss starts to increase, even if training loss is still decreasing.



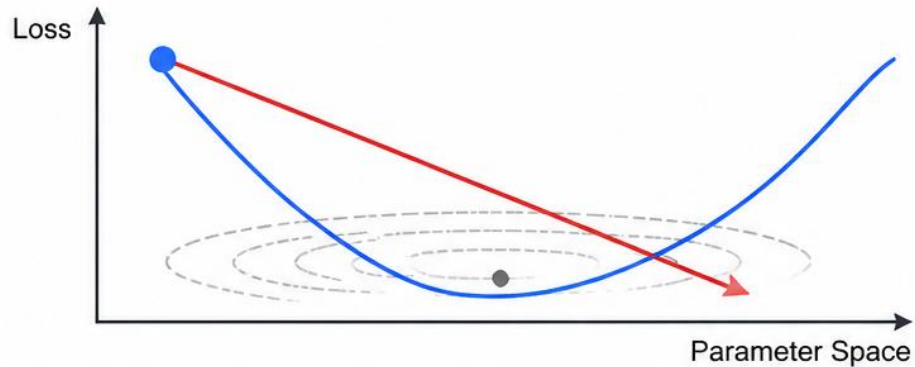
Gradient Clipping

- Mainly used in RNNs and LSTMs.
- Cap the maximum value of gradients to prevent the "Exploding Gradient" problem.

$$\text{Clip factor} = \frac{\text{clip threshold}}{\text{gradient norm}}$$

Gradient Clipping

Without Gradient Clipping

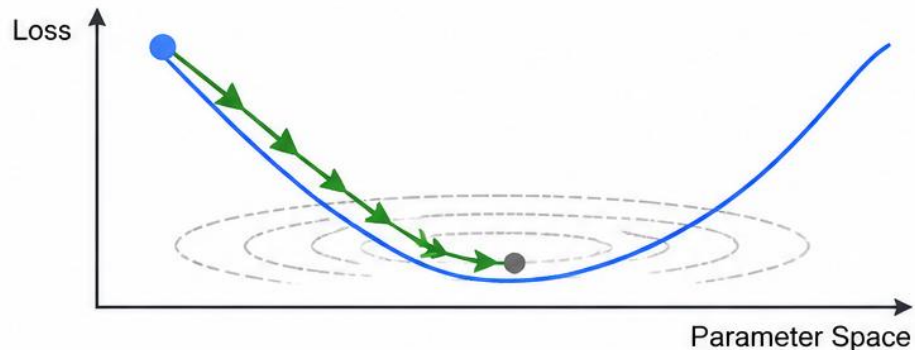


- Starting point
- Large gradient update
- Loss landscape
- - - Contours (level sets)
- Minimum

Problem:

Large gradients cause overshooting and unstable training.

With Gradient Clipping



- Starting point
- Clipped gradient update (limited step size)
- Loss landscape
- - - Contours (level sets)
- Minimum

Benefit:

Clipping gradients to a maximum norm prevents overshooting and leads to stable convergence.

How it works

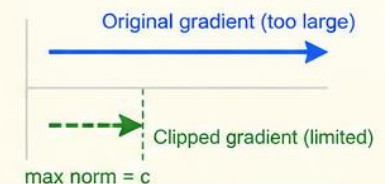
If the gradient norm is larger than a threshold c , we rescale it:

$$g_{\text{clipped}} = g \times \min\left(1, \frac{c}{\|g\|}\right)$$

g : original gradient

$\|g\|$: gradient norm (e.g., L2 norm)

c : clipping threshold



Summary and Best Practices

- Use Adam as a starting point.
- Always use Batch/Layer Norm for deep networks.
- Apply Dropout for regularization if the model is overfitting.
- Monitor loss curves to adjust learning rate schedules.

Practical Application: Mini-project

- **Task:** Students build a basic convolutional neural network (CNN) for image classification (e.g., CIFAR-10 or Fashion-MNIST) and apply optimization techniques to improve results.
- **Detail:**
 - Baseline setup does not have optimization techniques.
 - Make an effective comparison by adding Batch Normalization to stabilize the training process and Dropout to combat overfitting.
 - Experiment with different optimization algorithms such as SGD, Momentum, and Adam to observe the convergence rate.
 - Make a comparison table